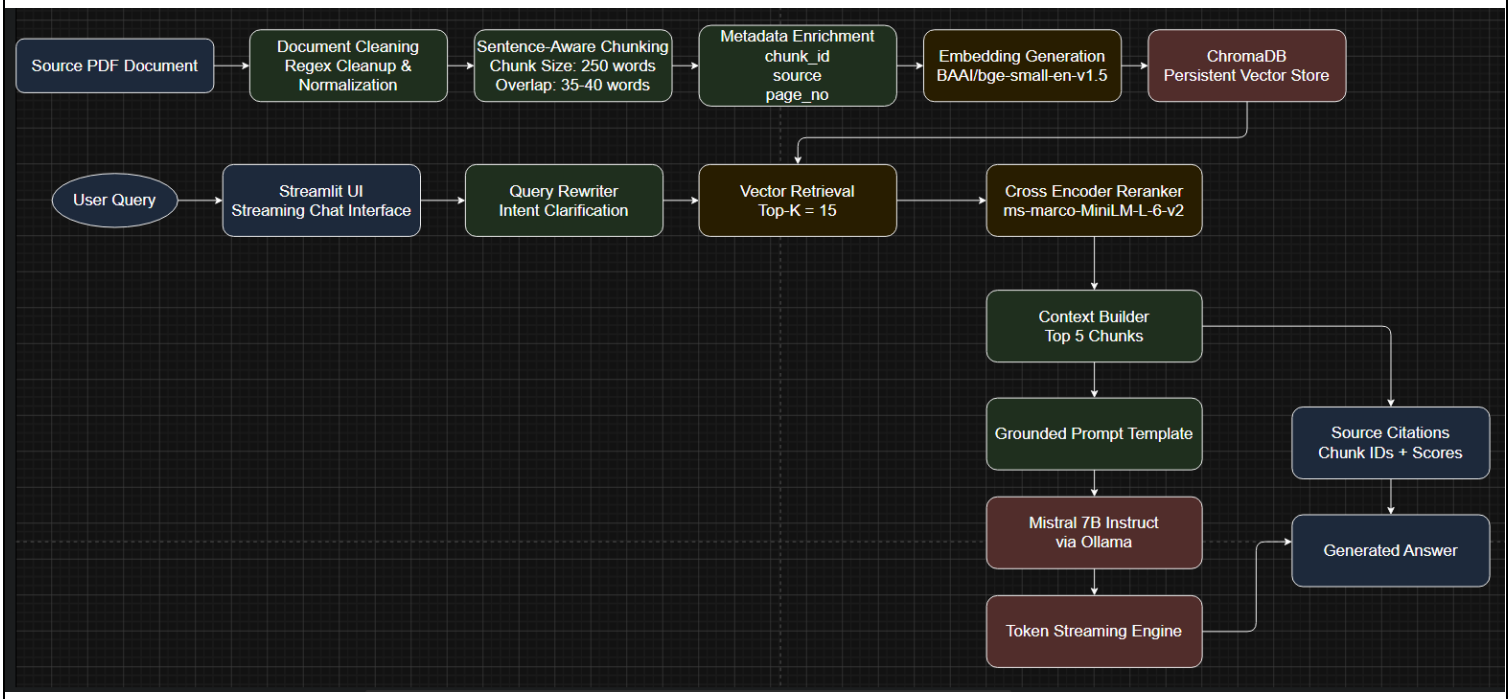
RAG Chatbot Architecture & Evaluation Report (Demo Link)

Architecture Diagram:



1. Document Structure and Chunking Logic

Document Loading and Cleaning

The application implements a robust document preprocessing pipeline designed for PDF documents, particularly policy and legal texts.

Documents are loaded using **PyMuPDF (fitz)**, which extracts text page by page. The extracted text undergoes a cleaning process to remove formatting artifacts that could negatively affect retrieval quality.

The cleaning process includes:

- Removal of standalone page numbers and “Page X of Y” patterns using regular expressions.
- Collapsing multiple spaces into a single space.
- Normalizing excessive consecutive newlines to a maximum of two.
- Trimming trailing whitespace from each line.

RAG Chatbot Architecture & Evaluation Report (Demo Link)

Chunking Strategy

The cleaned document is split into overlapping chunks using LangChain's **RecursiveCharacterTextSplitter**, ensuring semantic coherence while maintaining optimal chunk sizes for embedding generation.

Configuration:

- **Chunk Size:** 1,000 characters (approximately 250 words)
- **Chunk Overlap:** 150 characters (approximately 40 words)
- **Separators Used:**
  - Paragraph breaks (`\n\n`)
  - Line breaks (`\n`)
  - Sentence terminators (`.`, `!`, `?`)
  - Semicolons (`;`)
  - Spaces

Separators are evaluated in descending order of preference to preserve paragraph and sentence structure wherever possible.

Metadata Management

Each chunk is stored together with associated metadata:

- `chunk_id`
- `source` file name
- `word_count`
- `char_count`

This metadata improves traceability and enables source attribution during answer generation.

2. Embedding Model and Vector Database

Embedding Model

The system uses **BAAI/bge-small-en-v1.5** to generate dense vector embeddings for semantic retrieval.

Specifications:

- Embedding Dimension: 384

RAG Chatbot Architecture & Evaluation Report (Demo Link)

Rationale

The BGE-small model provides an excellent balance between retrieval performance and computational efficiency. Despite its lightweight footprint, it delivers strong semantic search capabilities and is well-suited for local deployment environments.

Vector Database

Embeddings are stored in **ChromaDB** using a persistent local storage configuration.

Configuration:

- Distance Metric: Cosine Similarity
- Storage Type: Persistent Chroma Client
- Indexing Strategy: Batch insertion (100 chunks per batch)

Batch insertion helps reduce memory spikes and improves indexing stability during document ingestion.

---

3. Prompt Format and Generation Logic

Generation Model

The application uses **Mistral-7B-Instruct**, served locally through **Ollama**.

Configuration:

- Context Window: 2,048 tokens
- Temperature: 0.1

The low temperature setting encourages deterministic, factual, and grounded responses while minimizing hallucinations.

Two-Stage Retrieval Pipeline

To maximize retrieval quality, the system employs a two-stage retrieval architecture.

Stage 1: Semantic Vector Search

ChromaDB retrieves the top 15 candidate chunks using cosine similarity.

- Retrieval Top-K: 15

Stage 2: Cross-Encoder Reranking

Retrieved candidates are reranked using:

cross-encoder/ms-marco-MiniLM-L-6-v2

RAG Chatbot Architecture & Evaluation Report (Demo Link)

The reranker evaluates query-chunk relevance and selects the most informative results.

- Reranked Top-N: 5

Only the highest-ranked chunks are forwarded to the language model.

Prompt Template

The retrieved context is injected into a grounded instruction prompt designed to prevent hallucinations.

```
You are a knowledgeable and precise assistant specializing in policy and lega
l documents.

Your task is to answer the user's question using ONLY the information found i
n the provided context.

Rules:

1. Base your answer strictly on the provided context.
2. If the context does not contain enough information, respond with:
   "The provided documents do not contain sufficient information to answer th
is question."
3. Do not fabricate or infer beyond what is explicitly stated.
4. Be clear, concise, and structured.
5. Use bullet points when listing information.

--- CONTEXT START ---

[Section 1]
<chunk text>

[Section 2]
<chunk text>

--- CONTEXT END ---

Question:
<user query>

Answer:
```

This prompt ensures that responses remain grounded in retrieved evidence and provides a safe fallback for unsupported queries.

RAG Chatbot Architecture & Evaluation Report (Demo Link)

4. Example Queries and Responses

Success Case 1: Direct Information Retrieval

Query

> What personal data is collected from users?

Response

> Based on the provided documents, the personal information collected from users includes their name, email address, and usage data when they register or use the services.

Analysis

The reranker correctly prioritized the chunk discussing data collection practices. The model accurately summarized the information without introducing unsupported details.

Success Case 2: Multi-Constraint Retrieval

Query

> How long is user data retained?

Response

> User data is retained for the duration necessary to fulfill the purposes outlined in this policy unless a longer retention period is required by law.

Analysis

The model successfully extracted the retention policy and avoided hallucinating specific retention periods that were not explicitly stated.

Success Case 3: User Rights and Procedures

Query

> Can I request deletion of my data?

Response

> Yes, you have the right to request the deletion of your personal data by contacting the privacy team.

Analysis

RAG Chatbot Architecture & Evaluation Report (Demo Link)

The response correctly identifies the user's rights and the required procedure based on the retrieved context.

Failure / Edge Case: Out-of-Domain Query

Query

> What is quantum physics?

Response

> The provided documents do not contain sufficient information to answer this question.

Analysis

This represents a successful failure case. The retrieval pipeline could not find relevant supporting information, and the prompt constraints prevented hallucination.

---

5. Limitations, Hallucinations, and Performance Notes

Hallucination Mitigation

Hallucinations are minimized through a combination of:

- Low temperature (0.1)
- Explicit grounding instructions
- Strict fallback responses
- Retrieval-based context injection

However, if the source document contains conflicting or ambiguous clauses, the model may generate answers that reflect those ambiguities.

Model and Hardware Limitations

Context Window Constraints

The 2,048-token context window limits the amount of information available to the model during generation.

If relevant information spans more than five retrieved chunks, the model may not receive all necessary context simultaneously.

RAG Chatbot Architecture & Evaluation Report (Demo Link)

Complex Reasoning

Although Mistral-7B-Instruct performs well for retrieval-augmented question answering, it may struggle with:

- Multi-hop reasoning
- Deep legal interpretation
- Cross-document synthesis

Larger models such as GPT-4 generally perform better in these scenarios.

Latency and Response Time

Running the entire pipeline locally introduces computational overhead.

Time to First Token (TTFT)

The following operations contribute to initial latency:

- Query embedding generation
- ChromaDB retrieval
- Cross-encoder reranking
- Prompt construction

Among these, reranking is typically the most computationally expensive step.

Generation Speed

Response generation speed depends heavily on the available hardware.

On systems with limited GPU memory (e.g., 4 GB VRAM), token generation may be relatively slow.

To improve perceived responsiveness, the application streams generated tokens incrementally through the Streamlit interface, allowing users to see responses as they are produced.

Conclusion

The implemented RAG chatbot successfully combines semantic retrieval, cross-encoder reranking, and grounded language generation to provide accurate document-based question answering. The architecture prioritizes retrieval quality, factual consistency, and transparency through source-aware responses and streaming output, making it well-suited for policy and legal document analysis.